BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 004 · Denary → floating point

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Choose a type

Q2. Convert a hexadecimal byte

Q3. Use two’s complement

Q4. Explain a fixed-width shift

// **BLACK PEN** Answer all questions first. /// **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — Mantissa

Recall the term — Exponent

Recall the term — Binary point

Recall the term — Normalisation

Recall the term — Precision

Recall the term — Range

New learning

Binary fractions continue right of the point — CODE:
6.5 = 4 + 2 + ½
= 110.1_2

0.375 = ¼ + ⅛
= 0.011_2

Floating point uses two signed fields — The mantissa stores the signed significant fraction. The exponent stores the signed power-of-two scale.

Normalise the mantissa — For a non-zero value, a positive normalised mantissa begins 01 ; a negative one begins 10 . — If redundant 00 or 11 sign bits remain, shift the mantissa left and decrease the exponent by the same number of places.

Precision and range — More mantissa bits improve precision . More exponent bits increase range .

Worked example

Represent +6.5 in normalised floating-point form

1. Write 6.5 in binary — CODE:
6.5 = 110.1_2 — The whole-number part is 110 ; the 0.5 fractional part is .1 .

2. Move the binary point — CODE:
110.1_2
= $0.1101_2 \times 2^3$ — The binary point moves three places left, so the exponent is 3 .

3. Write the two fields — CODE:
Mantissa: 01101000
Exponent: 0011

// **BLACK PEN** Answer all questions first. /// **GREEN PEN** Then: tick and rewrite mistakes.

Q5. Mantissa 10000000 (1); exponent 1111 (1). $-1 \times 2^{-1} = -0.5$. 11000000 0000 has the correct value but is not normalised.

Exit Ticket · Model answer

True. More mantissa bits give greater precision to the represented value.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Real / float: a number with a fractional part.

Q2. 00111100 = 32 + 16 + 8 + 4 = 60.

Q3. +26 = 00011010; invert → 11100101; add 1 → 11100110 .

Q4. 01100000 = 96. The leading 1 is discarded; 176 × 2 = 352 exceeds the byte's maximum 255.

You Do pages · Independent questions

Q1. CODE:
10.5 = 1010.1_2 = $0.10101_2 \times 2^4$ → 01010100 0100
$0.375 = 0.011_2 = 0.11_2 \times 2^{-1}$ → 01100000 1111
Each: correct binary magnitude/scaling working (1); both final fields (1).

Q2. CODE:
−10.5: invert 01010100 → 10101011; +1 → 10101100
Final: 10101100 0100
−0.375: invert 01100000 → 10011111; +1 → 10100000
Final: 10100000 1111
Each: positive binary magnitude/scaling (1), correct negative mantissa (1), complete normalised fields (1).

Q3. CODE:
00001011 0111
00010110 0110
00101100 0101
01011000 0100
Three left shifts (1); exponent reduced by 3 (1); mantissa 01011000 (1); exponent 0100 (1). Check: original 11/128 × 128 = 11; final 88/128 × 16 = 11.

Q4. The mantissa was doubled but the scale stayed the same, so the value doubled (1). Reduce exponent 3 to 2 / 0010 (1). Correct fields: 01010000 0010 (1). Normalisation removes redundant sign bits to make full use of the available significant bits (1).

// BLACK PEN Answer all questions first. //✓ GREEN PEN Then: tick and rewrite mistakes.

Stored: 01101000 0011 — Check: 0.8125 × 8 = 6.5 .

We Do · Practise with guidance

2. A small fraction: 0.15625

Use the method from I Do. Give the final value in normalised floating-point form using an eight-bit two's-complement mantissa and a four-bit two's-complement exponent.

Build the answer

−2 in four-bit two's complement is 1110 .

3. Negative value: −6.5

Check: −0.8125 × 8 = −6.5.

4. Write the pattern in normalised floating-point form

5. Represent −0.5 in normalised floating-point form

$-1 \times 2^{-1} = -0.5$.

Represent −15.75 in normalised floating-point form

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Represent positive values
Represent (a) 10.5 and (b) 0.375 in normalised floating-point form using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Show the binary conversion and both fields. [2 each]
Adapted from the OCR 2023 conversion task: values changed; classroom marks.

Question 2

Represent negative values
Represent (a) −10.5 and (b) −0.375 in normalised floating-point form using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Show the positive binary value, the mantissa negation and the final fields. [3 each]
Adapted from OCR June 2023 Q3(c): values changed; classroom marks.

Question 3

Normalise the fields
Write 00001011 0111 in normalised floating-point form using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Show the shifts and exponent adjustment.
Paraphrased exam question; original pattern and marks retained. OCR June 2024, Q4(d), page 15

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Question 4

Diagnose the mistake
A pupil is using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent. They change 00101000 0011 to 01010000 0011 and call it normalisation. Explain why the value changed [1], correct the exponent [1], give the complete correct pattern [1], and explain the purpose of normalisation [1].

Question 5

Handle a boundary value
Represent −0.5 in normalised floating-point form using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the mantissa [1] and exponent [1].

Exit Ticket · Answer before checking

More mantissa bits improve floating-point precision.